

บทที่ 3

Enterprise Platforms

3.1. Enterprise Platforms

สำหรับ Enterprise Platforms นั้นจะสามารถแบ่งได้เป็น 2 ประเภทใหญ่ๆ ได้แก่ คอทเน็ต(.NET) และเจทูอีอี(J2EE)

3.1.1. เอเอสพีคอทเน็ต (ASP.NET) พัฒนาด้วยภาษาในกลุ่มคอทเน็ตเทคโนโลยี (.NET Technology) เช่น ซีชาร์ป (C#) , วีบี (VB) , ...

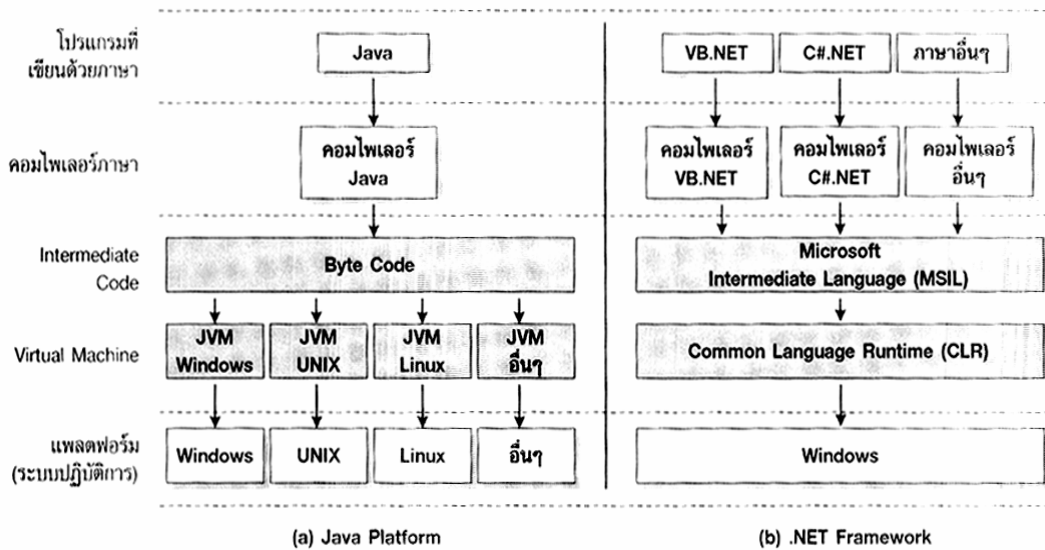
3.1.2. เจทูอีอี แบ่งเป็น EJB , JSP , Java Servlet , ทั้งหมดใช้จาวา (Java)

3.2. การเปรียบเทียบระหว่างข้อดีและข้อเสียของ .NET และ J2EE

	คอทเน็ต	เจทูอีอี
ข้อดี	1.) คอทเน็ตเฟรมเวิร์ค(.NET Framework) ฟรี 2.) ไม่จำกัดภาษา เลือกตามความถนัดในการพัฒนาได้ (language neutral) แต่ปฏิบัติงานได้บนระบบปฏิบัติการวินโดวส์เท่านั้น (Windows Centric) หรือที่เรียกว่า “Write any language, run on Windows” 3.) มีคุณสมบัติโอโอพีอยู่เพราะภาษาในคอทเน็ต (.NET) จะยึดคุณสมบัติโอโอพี 4.) ซ่อนซอร์สโค้ด (Code Behind) 5.) ง่ายต่อการพัฒนา เนื่องจากมีวิซวลสตูดิโอ คอทเน็ต (Visual Studio .NET) ซึ่งน่าจะเป็นไอดีอี (IDE) ที่ดีที่สุด 6.) มีเอ็มเอสดีเอ็น (MSDN) ซึ่งง่ายต่อการทำความเข้าใจหากสงสัยเรื่องใดๆ 7.) ง่ายต่อการศึกษา และ ทำความเข้าใจ เขียนง่ายกว่า และเซอร์เวอร์ก็สร้าง (Maintain) ง่ายกว่า	1.) เจทูอีอีแจกฟรี 2.) มีเทคโนโลยีให้เลือกใช้มากมายตามความเหมาะสม 3.) มีคุณสมบัติของโอโอพี (OOP) ตามแบบฉบับจาวา

ข้อเสีย	1.) รันได้เฉพาะบนไมโครซอฟท์แพลตฟอร์ม (Microsoft Platform) 2.) Visual Studio.NET มีราคาแพงมาก (แต่ก็มี Sharp developer และ Web Matrix ให้ใช้)	1.) ถึงแม้ว่าจะเป็นโอเพนซอร์ส (Open Source) และแจกฟรี แต่ระบบที่จะรันต้องมีต้นทุนสูงกว่า ไมโครซอฟท์แพลตฟอร์ม รวมถึงบุคลากรต้องมีความรู้พอสมควร จึงถือเป็นการเพิ่มต้นทุนแก่องค์กรในระยะยาว 2.) ไอดีอีที่ใช้ในการพัฒนายังไม่ดีเท่าที่ควร 3.) ยากต่อการศึกษาและทำความเข้าใจ คือเขียนยากกว่าและสร้างเซิร์ฟเวอร์ (Server Maintain) ยากกว่า ทั้งเจเอสพี (JSP) และโค้ดของคอตเน็ตจะถูก Compile เหมือน Application ทั่วไปแต่ในโค้ดของเจเอสพีจะถูกคอมไพล์เป็น HTTP Servlet Class ในตอนที่เว็บถูกเรียกใช้ครั้งแรก (ทำให้มันเฉพาะครั้งแรก) ซึ่งคอตเน็ตจะถูกคอมไพล์เป็นแอสเซมบลี (DLL แบบใหม่ของคอตเน็ต) เก็บไว้เลย ทำให้จาวาช้ากว่า คอตเน็ต

ตารางที่ 3.1 การเปรียบเทียบระหว่างข้อดีและข้อเสียของคอตเน็ต (.NET) และเจทูอีอี (J2EE)



รูปที่ 3.1 การเปรียบเทียบคอตเน็ตเฟรมเวิร์ค (.NET Framework) และจาวาแพลตฟอร์ม (Java Platform)

3.3 คอตเน็ตเฟรมเวิร์ค (.NET Framework)

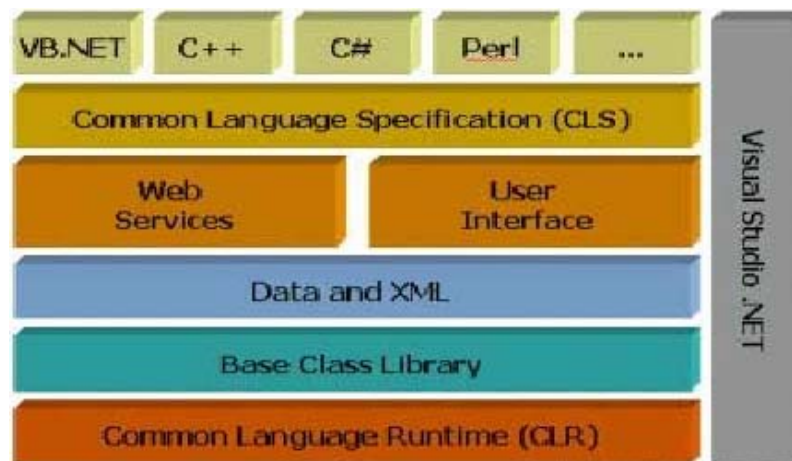
ไม่มีใครชอบพท์ต้องการที่จะสร้างอะไรที่เป็นมาตรฐานขึ้นมา เพื่อให้ทุกสิ่งทุกอย่าง เพื่อให้ทุกสิ่งทุกอย่างสามารถติดต่อสื่อสารกันได้หมด โดยคิดค้นระบบซึ่งหมายความว่าให้เป็นระบบมาตรฐาน ระบบนี้คือคอตเน็ตเฟรมเวิร์ค ซึ่งระบบนี้ไม่ใช่ระบบปฏิบัติการ (OS) แต่เปรียบเสมือนโปรแกรมหนึ่งที่จะสามารถสร้างสภาวะแวดล้อมหนึ่ง ซึ่งสามารถทำงานในระบบคอตเน็ตนี้ได้

คอตเน็ตเฟรมเวิร์ค คือ โครงสร้างการพัฒนาโปรแกรมคอมพิวเตอร์ ที่ถูกออกแบบมาเพื่อให้อำนวยความสะดวกในการพัฒนาโปรแกรมสมัยใหม่ ที่ใช้งานในระบบเครือข่าย (Internet, Intranet, Mobile Devices, ฯลฯ) Bill Gates และ Steve Ballmer ได้บรรยายสรุปวิสัยทัศน์ ที่เกี่ยวกับคอตเน็ตเอาไว้ 3 ข้อหลัก ๆ ได้แก่

- 1) การพัฒนาโปรแกรมในรูปแบบของเว็บเซอร์วิส จะเป็นหัวใจหลักในการขับเคลื่อนโปรแกรมต่างๆ ที่ใช้งานบนเว็บเซอร์วิสอินเทอร์เน็ตจะช่วยให้การติดต่อสื่อสารระหว่างแอปพลิเคชันบนอินเทอร์เน็ตนั้นง่ายขึ้น และเป็นระบบมากยิ่งขึ้น
- 2) เว็บเซอร์วิสขั้นพื้นฐานเช่น การตรวจสอบผู้ใช้ที่ล็อกอิน (log in) เข้าสู่ระบบ จะถูกพัฒนาให้เป็นมาตรฐาน และสามารถนำไปใช้ได้ทั่วไปบนอินเทอร์เน็ต
- 3) พีซี (PC= desktop, notebook) และเครื่องมือแบบเคลื่อนที่ (Mobile Device) ที่ต่อเชื่อมกับอินเทอร์เน็ตได้ เช่น PDA และ โทรศัพท์มือถือ จะมีบทบาท และประโยชน์มากขึ้นไปอีก เมื่อสามารถติดต่อใช้งานโปรแกรมต่าง ๆ บนอินเทอร์เน็ตได้

ไมโครซอฟท์จึงได้พัฒนารูปแบบการพัฒนาในระบบคอมพิวเตอร์ขึ้นมา ซึ่งเรียกว่า ดอทเน็ตเฟรมเวิร์คนั่นเอง (อันที่จริงแล้วไมโครซอฟท์ไม่ได้เป็นผู้คิดค้นเรื่องพวกนี้ขึ้นมาแต่เพียงผู้เดียว อย่าเข้าใจผิด สิ่งต่าง ๆ เหล่านี้มีผู้คิดค้นพัฒนาจากหลายบริษัท หลายหน่วยงาน ยกตัวอย่างเช่น ซันไมโครซิสเต็ม (Sun Microsystems), ไอบีเอ็ม (IBM), ฯลฯ หากแต่ไมโครซอฟท์นำแนวคิดเหล่านั้นมาออกแบบให้อยู่ในรูปแบบที่สามารถต่อเชื่อมกันได้ง่ายขึ้น เป็นระบบมากขึ้น) เพราะฉะนั้น ถ้าจะให้นิยามคำว่าดอทเน็ตคงไม่สามารถชี้เฉพาะได้ว่าดอทเน็ตคืออะไร เพราะจริง ๆ แล้วดอทเน็ตประกอบไปด้วยส่วนประกอบต่าง ๆ หลายส่วนด้วยกัน ส่วนประกอบเหล่านี้ถูกออกแบบมาเพื่อให้ทำงานได้เข้ากันได้ดียิ่งขึ้น

ในอนาคตไมโครซอฟท์ก็หวังที่จะนำเอาระบบนี้ไปติดตั้งลงบนอุปกรณ์ทุกชนิด เพื่อให้ทุกอุปกรณ์ทุกอย่างมีระบบฯหนึ่งที่เหมือนกันหมด โดยดอทเน็ตเฟรมเวิร์ค นั้นมี ส่วนประกอบภายในแบ่งออกเป็นชั้นดังนี้คือ



รูปที่ 3.2 โครงสร้างของดอทเน็ตเฟรมเวิร์ค

3.3.1. Programming Language

เป็นภาษาที่ใช้สร้างโปรแกรมซึ่งสามารถทำงานได้ภายใต้สถานะของดอทเน็ต โดยไมโครซอฟท์ได้เปิดตัวภาษาหลักๆที่จะใช้พัฒนาบนดอทเน็ตนี้มีจำนวน 3 ภาษาด้วยกันคือ

- 1.) ซีชาร์ป © เป็นภาษาใหม่ที่ไมโครซอฟท์พัฒนามาจากภาษาซีพลัสพลัส (c++) กับ ภาษาจาวา (JAVA) เป็นหลัก
- 2.) วิบีดอทเน็ต (VB.NET = Visual Basic .NET หรือ Visual Basic เวอร์ชัน 7.0 นั่นเอง) เป็นภาษาที่พัฒนามาจาก Visual Basic ในเวอร์ชัน 6.0
- 3.) เจสคริปต์ดอทเน็ต (Jscript.NET) เป็นภาษาที่พัฒนามาจากเจสคริปต์ (Jscript) ซึ่งเป็นจาวาสคริปต์ (JavaScript) ในเวอร์ชันของไมโครซอฟท์

3.3.2. Base Classes Library :

ไลบรารี (Library) นั้นเปรียบเสมือนชุดคำสั่งสำเร็จรูปย่อยๆ ที่เพิ่มเข้ามา ซึ่งส่วนใหญ่จะเป็นชุดคำสั่งที่ต้องใช้งานอยู่เป็นประจำ ดังนั้นจึงมีผู้คิดค้นเพื่ออำนวยความสะดวกในการเขียนโปรแกรม ซึ่งไลบรารีในภาษาต่างๆ ส่วนใหญ่จะอยู่ในรูปแบบไฟล์ incoould แต่ถ้าเป็น ASP สิ่งที่เป็น ไลบรารีก็คือ คอมโพเนนท์ต่างๆ นั้นเอง ซึ่งภายในระบบคอตเน็ตจะสร้างสิ่งที่เรียกว่าเป็นไลบรารี พื้นฐานขึ้น ทำให้ไม่ว่าจะใช้ภาษาใดในการพัฒนาโปรแกรมก็สามารถที่จะเรียกใช้ไลบรารีที่เป็นตัวเดียวกันได้หมด

3.3.3. Common Language Runtime (CLR) :

นับเป็นสิ่งสำคัญแทบจะที่สุดของระบบคอตเน็ตนี้ก็ได้ เพราะ CLR ที่ว่านี้มีหน้าที่ทำให้โปรแกรมที่เขียนขึ้นมาด้วยภาษาต่างๆ กัน กลายเป็นภาษารูปแบบมาตรฐานเดียวกัน ทั้งหมด เราเรียกภาษาที่ว่านี้ว่า Intermediate language (IL) ซึ่งเมื่อต้องการที่จะรันโปรแกรมใด CLR ที่ว่านี้จะตรวจสอบเครื่องที่รันว่ามีสถานะแวดล้อมการทำงานเช่นใดหลังจากนั้นก็คอมไพล์เป็นโปรแกรมที่เหมาะสมต่อการทำงานของเครื่องนั้น ทำให้เราสามารถใช้งานโปรแกรม ต่างๆ ได้อย่างมีประสิทธิภาพสูงสุดในแต่ละเครื่อง

3.3.4. ข้อดีของคอตเน็ตเฟรมเวิร์ค

ประโยชน์และข้อดีของคอตเน็ตเฟรมเวิร์คนั้นพอจะสรุปออกมาได้เป็นข้อๆ ดังนี้

1.) เป็นระบบที่มีไลบรารีที่เป็นมาตรฐานเดียวกัน : เนื่องจากมีไลบรารีที่เป็นมาตรฐานเดียวกันทั้งหมดทำให้เราไม่ต้องกังวลว่าภาษาที่ใช้เขียนนั้นมีไลบรารีตัวนั้นตัวนี้หรือไม่ รวมทั้งไม่ต้องคอยกังวลว่าถ้าใช้ไลบรารีของภาษาหนึ่งแล้วอีกภาษาหนึ่งจะไม่มีไลบรารีตัวนั้น

2.) ไม่ขึ้นกับระบบปฏิบัติการ (OS) : เนื่องจากระบบปฏิบัติการ ที่แต่ละบุคคลหรือองค์กรใช้นั้นย่อมไม่เหมือนกัน แต่ภายในคอตเน็ตเฟรมเวิร์คจะไม่มีปัญหานี้ของเพียงแค่มีคอตเน็ตเฟรมเวิร์คก็จะทำให้สามารถใช้งาน โปรแกรมต่างๆ ได้ ซึ่งเป็นข้อดีตรงที่เราจะสามารถใช้โปรแกรมต่างๆ ได้ทุกระบบปฏิบัติการ

3.) ใช้ในการพัฒนาได้ทุกภาษา : ทำให้เราไม่ต้องคอยมาศึกษาภาษาใหม่ๆ เมื่อต้องการสร้างโปรแกรมในแต่ละครั้ง นอกจากนั้นเรายังสามารถเลือก ใช้ภาษาที่เราถนัดที่สุดในการพัฒนาโปรแกรมต่างๆ ได้ด้วย

4.) มีการควบคุมสิ่งแวดล้อมในการทำงานเป็นอย่างดี : เนื่องจากเป็นระบบที่เป็นมาตรฐาน ทำให้การควบคุมจัดสรรระบบต่างๆ ทำได้ง่ายขึ้น ไม่ว่าจะเป็นการจัดสรรหน่วยความจำ ด้านการใช้งานเครื่องก็มีความรวดเร็วมากขึ้น ลดโอกาสที่เครื่องจะแฮงค์ได้เป็นอย่างดี

5.) ความปลอดภัยที่มีมากขึ้น คอเน็คเฟรมเวิร์คสามารถกำหนดสิทธิ์การใช้งานหรือ การอนุญาตให้ใช้ (Permission) ของผู้ใช้งานได้มากขึ้นทำให้สามารถกำหนดว่า จะให้โปรแกรมในส่วนใดใช้งานได้หรือไม่ได้ แล้วแต่เฉพาะบุคคล

ทั้งหมดนี้เป็นเพียงประโยชน์ส่วนหนึ่งในแนวคิดของไมโครซอฟท์ที่กำลังจะพัฒนาให้สำเร็จเท่านั้น บางข้อทำได้สำเร็จแล้ว แต่บางข้อก็ยังไม่สำเร็จดังนั้นจึงต้องคอยรอดูว่า ฝันของไมโครซอฟท์จะเป็นจริงและสำเร็จได้เมื่อไร

3.4 ขั้นตอนการสร้างเว็บเซอร์วิส

1. เรียกใช้โปรแกรม Microsoft Visual Studio.NET 2003
2. เลือกที่ File > New > Project
3. ในหน้าต่างของ New Project เลือกเพิ่มเพลตเป็น ASP.NET Webservices แล้วกรอกพาธที่จะไปทำการเรียกใช้โปรแกรม (Location) แล้วกด “O.K”
4. โปรแกรมจะทำการ Generate Code ตัวอย่างให้อัตโนมัต ซึ่งจะอยู่ในไฟล์ .asmx ต่อไปนี้เป็นซอร์สโค้ดตัวอย่าง (source code) ของโปรแกรม ซึ่งเมื่อมีการ invoke จะมีแมสเสจ (message) คำว่า “Hello world ” เกิดขึ้น

```
using System;
using System.Collections;
using System.ComponentModel;
using System.Data;
using System.Diagnostics;
using System.Web;
using System.Web.Services;

namespace hello
{
    /// <summary>
    /// Summary description for Service1.
    /// </summary>
    public class Service1 : System.Web.Services.WebService
    {
        public Service1()
        {
            //CODEGEN: This call is required by the ASP.NET Web Services Designer
            InitializeComponent();
        }

        #region Component Designer generated code

        //Required by the Web Services Designer
        private IContainer components = null;

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
        }
    }
}
```

```

/// <summary>
/// Clean up any resources being used.
/// </summary>
protected override void Dispose( bool disposing )
{
    if(disposing && components != null)
    {
        components.Dispose();
    }
    base.Dispose(disposing);
}

#endregion

// WEB SERVICE EXAMPLE
// The HelloWorld() example service returns the string Hello World
// To build, uncomment the following lines then save and build the project
// To test this web service, press F5

[WebMethod]
public string HelloWorld()
{
    return "Hello World";
}
}

```

รูปที่ 3.3 ซอร์สโค้ดของตัวอย่าง

Service1

The following operations are supported. For a formal definition, please review the [Service Description](#).

- [HelloWorld](#)

This web service is using <http://tempuri.org/> as its default namespace.

Recommendation: Change the default namespace before the XML Web service is made public.

Each XML Web service needs a unique namespace in order for client applications to distinguish it from other services on the Web. <http://tempuri.org/> Web services that are under development, but published XML Web services should use a more permanent namespace.

Your XML Web service should be identified by a namespace that you control. For example, you can use your company's Internet domain name as Although many XML Web service namespaces look like URLs, they need not point to actual resources on the Web. (XML Web service namespaces

For XML Web services creating using ASP.NET, the default namespace can be changed using the WebService attribute's Namespace property. The an attribute applied to the class that contains the XML Web service methods. Below is a code example that sets the namespace to "http://microso

C#

```

[WebService(Namespace="http://microsoft.com/webservices/")]
public class MyWebService {
    // implementation
}

```

Visual Basic.NET

```

<WebService(Namespace:="http://microsoft.com/webservices/")> Public Class MyWebService
    ' implementation
End Class

```

For more details on XML namespaces, see the W3C recommendation on [Namespaces in XML](#).

For more details on WSDL, see the [WSDL Specification](#).

For more details on URIs, see [RFC 2396](#).

รูปที่ 3.4 ผลลัพธ์เมื่อรันโปรแกรมตัวอย่าง

```
<? xml version="1.0" encoding="utf-8" ?>  
<string xmlns="http://tempuri.org/">Hello World</string>
```

รูปที่ 3.5 ผลลัพธ์ของแมสเสจที่เป็นภาษา XML